FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

NI-NLM – Lecture 3

Training language models

Zdeněk Kasner

3 Mar 2026

Training neural networks

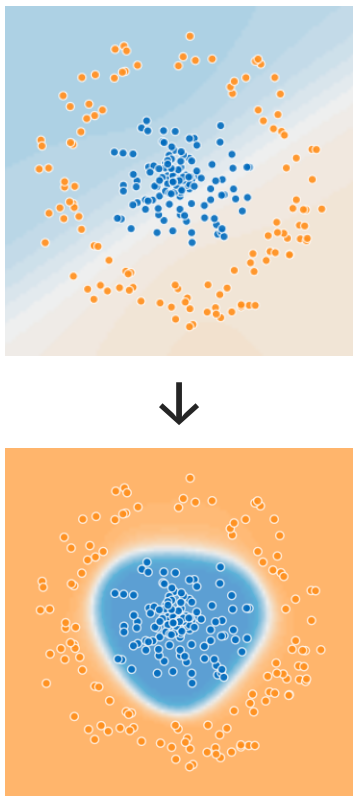
Training neural networks

source: https://playground.tensorflow.org

👩‍🍳 Recipe for supervised training of NNs

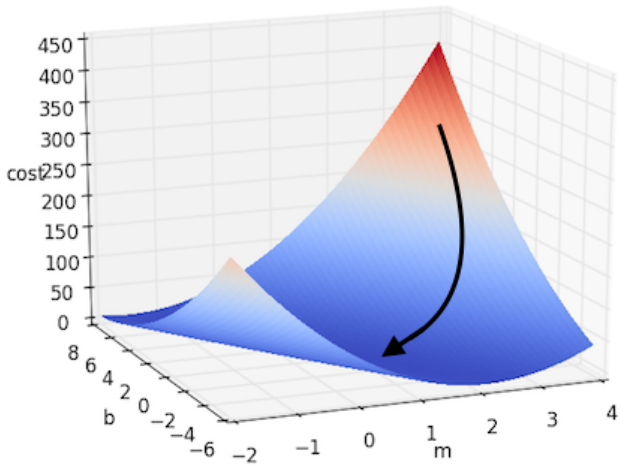
Repeat for each training example (x, y):

1. Let the model **predict** $\mathcal{M}(x, y) = \hat{y}$.
2. Compute a **loss function** $\mathcal{L}(\hat{y}, y)$: how wrong the model prediction $\hat{y}$ is.
3. Compute **gradients**: how each parameter of the model $\mathcal{M}$ contributed to the loss $\mathcal{L}$.
4. Apply **backpropagation**: update the model $\mathcal{M}$ parameters in the direction that reduces the loss $\mathcal{L}$.

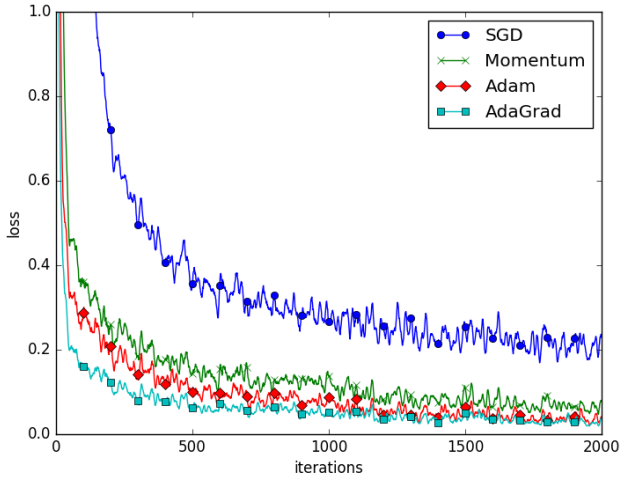


How does training a neural network look like?

source: https://playground.tensorflow.org



source: https://ml4a.github.io



source: https://kaeken.hatenablog.com/

Training a language model – closer look

source: Karpathy's microgpt

```
# Repeat in sequence
num_steps = 1000 # number of training steps
for step in range(num_steps):

    # Take single document, tokenize it, surround it with BOS special token
on both sides
    doc = docs[step % len(docs)]
    tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
    n = min(block_size, len(tokens) - 1)

    # Forward the token sequence through the model, building up the
computation graph all the way to the loss
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    losses = []
    for pos_id in range(n):
        token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        loss_t = -probs[target_id].log()
        losses.append(loss_t)
    loss = (1 / n) * sum(losses) # final average loss over the document
sequence. May yours be low.
```

```
    # Backward the loss, calculating the gradients with respect to all model
parameters
    loss.backward()

    # Adam optimizer update: update the model parameters based on the
corresponding gradients
    lr_t = learning_rate * (1 - step / num_steps) # linear learning rate
decay
    for i, p in enumerate(params):
        m[i] = beta1 * m[i] + (1 - beta1) * p.grad
        v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
        m_hat = m[i] / (1 - beta1 ** (step + 1))
        v_hat = v[i] / (1 - beta2 ** (step + 1))
        p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
        p.grad = 0

    print(f"step {step+1:4d} / {num_steps:4d} | loss {loss.data:.4f}",
end='\r')
```

Loss function

A **loss function** (or cost function) measures **how far off** the model's prediction is from the correct answer.

How do we measure that? Depends on our problem.

> **Question**
>
> Assume your model predicts a **real number**. How would you measure the error with respect to the ground truth value?

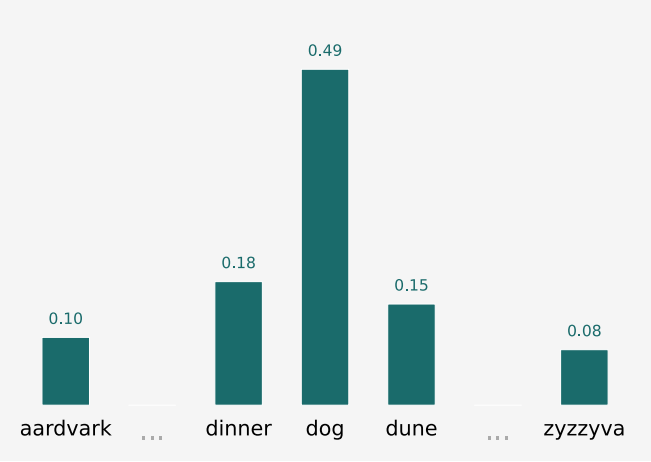
- `(true - predicted)`: bad idea, the errors may cancel out
- `abs(true - predicted)`: ok, but non-differentiable at 0 → training issues
- `(true - predicted)^2`: works well → smooth, penalizes large errors

Loss function: language modeling

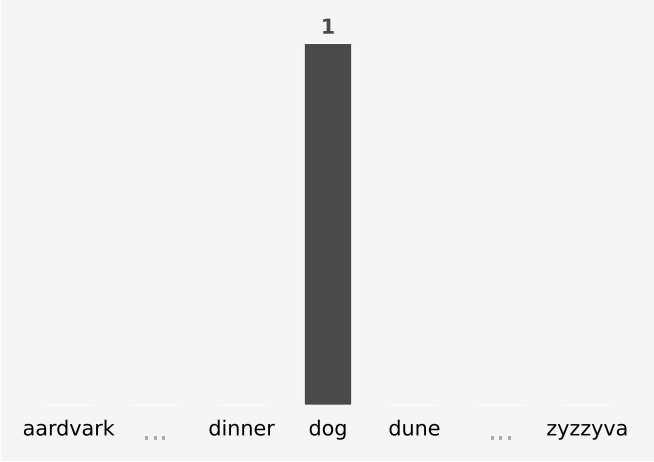
Question

Assume your model predicts a **probability distribution for the next word**. How would you measure the error with respect to the ground truth value?

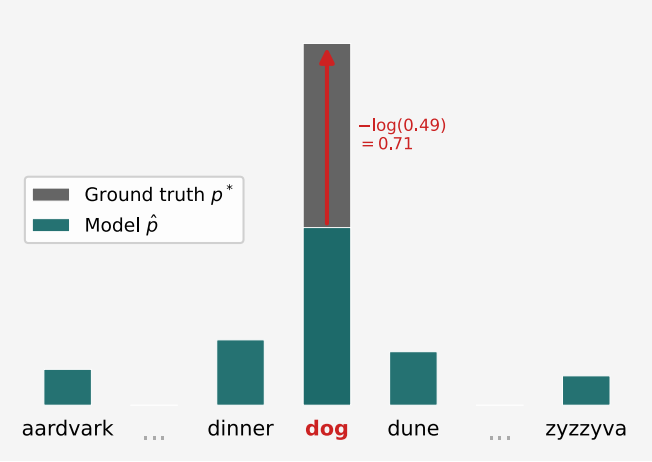
Model prediction



Ground truth



Model error



Loss function: language modeling

source: The Illustrated Transformer

What we train for:

Output Vocabulary:	a	am	I	thanks	student	<eos>
position #1	0.0	0.0	1.0	0.0	0.0	0.0
position #2	0.0	1.0	0.0	0.0	0.0	0.0
position #3	1.0	0.0	0.0	0.0	0.0	0.0
position #4	0.0	0.0	0.0	0.0	1.0	0.0
position #5	0.0	0.0	0.0	0.0	0.0	1.0

What we hope to get:

Output Vocabulary:	a	am	I	thanks	student	<eos>
position #1	0.01	0.02	0.93	0.01	0.03	0.01
position #2	0.01	0.8	0.1	0.05	0.01	0.03
position #3	0.99	0.001	0.001	0.001	0.002	0.001
position #4	0.001	0.002	0.001	0.02	0.94	0.01
position #5	0.01	0.01	0.001	0.001	0.001	0.98

Cross-entropy loss

The loss function used for the next word prediction task is called **cross-entropy**:

$$\mathcal{L} = -\sum_{c=1}^{C} y_c \cdot \log(\hat{y}_c)$$

where y_c is the true label (one-hot) and $\hat{y}_c$ is the predicted probability.

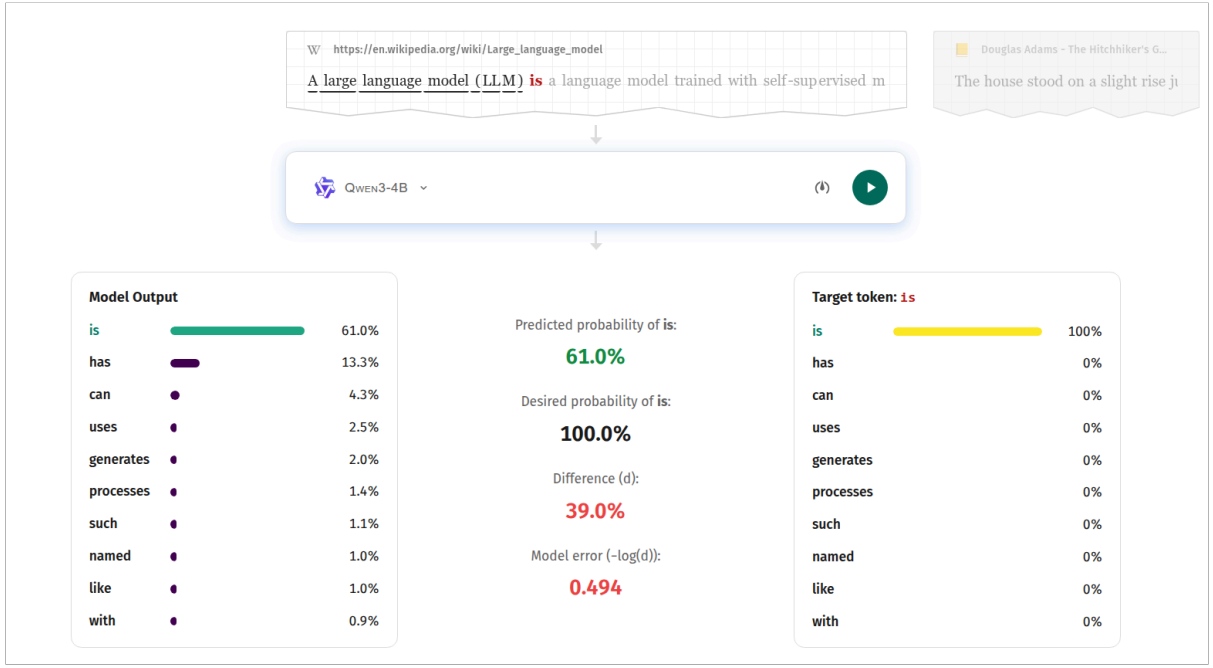
Intuition about cross-entropy for language modeling

- The model is supposed to predict the target token with **100% probability**.
- Any difference between these probabilities is considered as an **error**.
- The value of the loss is a **negative logarithm of the difference**. (why?)
- We ignore model predictions for any other tokens. (why?)

Cross-entropy in language models

Let's see it animated:

https://animatedllm.github.io/pretraining-simple



Descending the loss landscape

source: Hackernoon

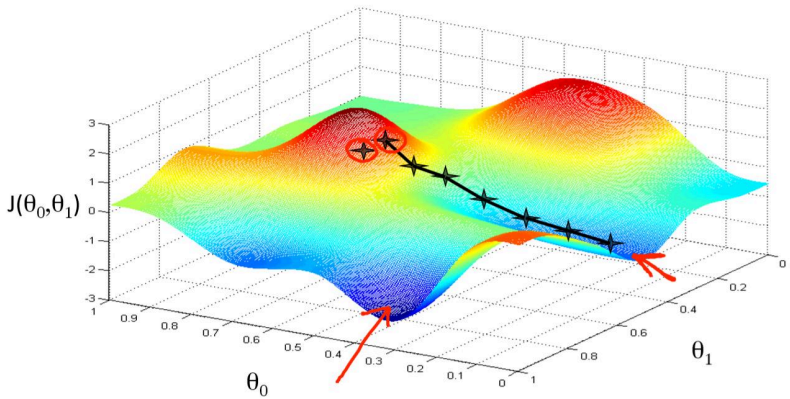
Our goal is to **minimize the loss** by iteratively **updating the model parameters** θ in the direction that reduces the loss $\mathcal{L}$:

Gradient descent as a math formula

$$\theta \leftarrow \theta - \eta \nabla \mathcal{L}(\theta)$$

This simply says: we compute the steepest ascent direction (∇) and go the opossite way.

- ∇ is the **gradient** of the loss w.r.t. each parameter.
- η is the **learning rate**.

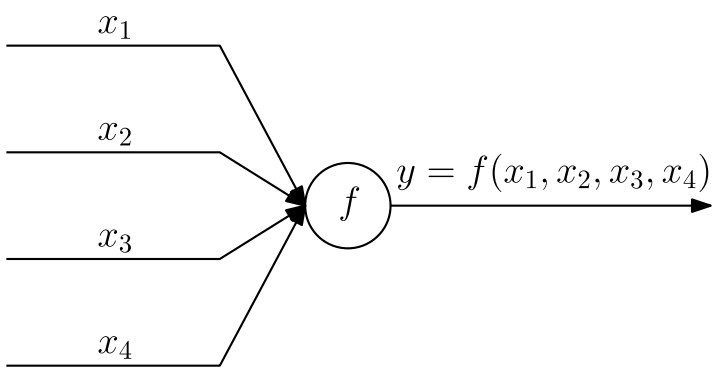


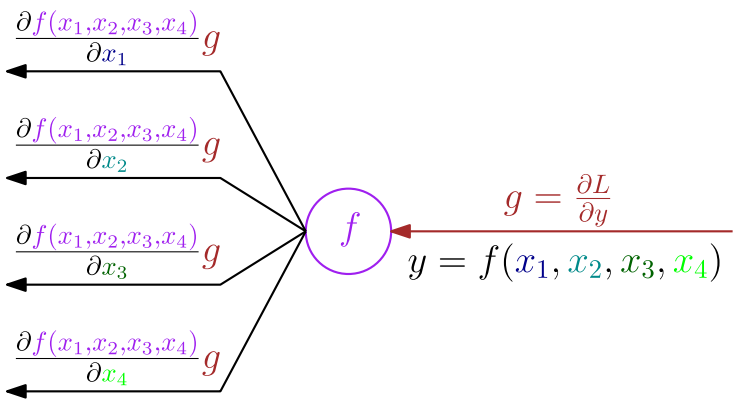
Computing gradient

source: NPFL138

Question

We ended the forward pass by computing the loss. How do we get the gradient?





We start by computing $\frac{\partial \mathcal{L}}{\partial y}$ → **the derivative of the loss with respect to the model's output probabilities** (the softmax outputs).

Computing gradient

Since our loss is computed as:

$$\mathcal{L} = -\log y_{\text{correct}},$$

then:

$$\frac{\partial \mathcal{L}}{\partial y_c} = \begin{cases} -\frac{1}{y_c} & \text{for the correct class} \\ 0 & \text{for all other classes} \end{cases}$$

Gradient vs. derivative

Gradient (∇) is how a (partial) **derivative** ∂ is called for multivariate functions (= functions operating on multi-dimensional inputs).

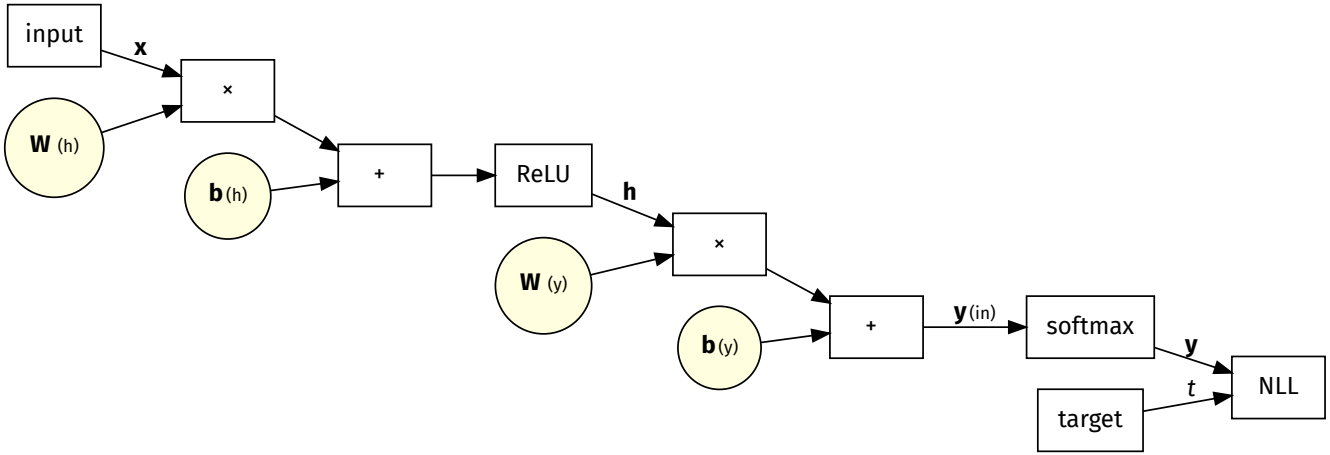
Backpropagation

source: NPFL129

Backpropagation = repeated application of the **chain rule** of derivatives for $f(g(x))$:

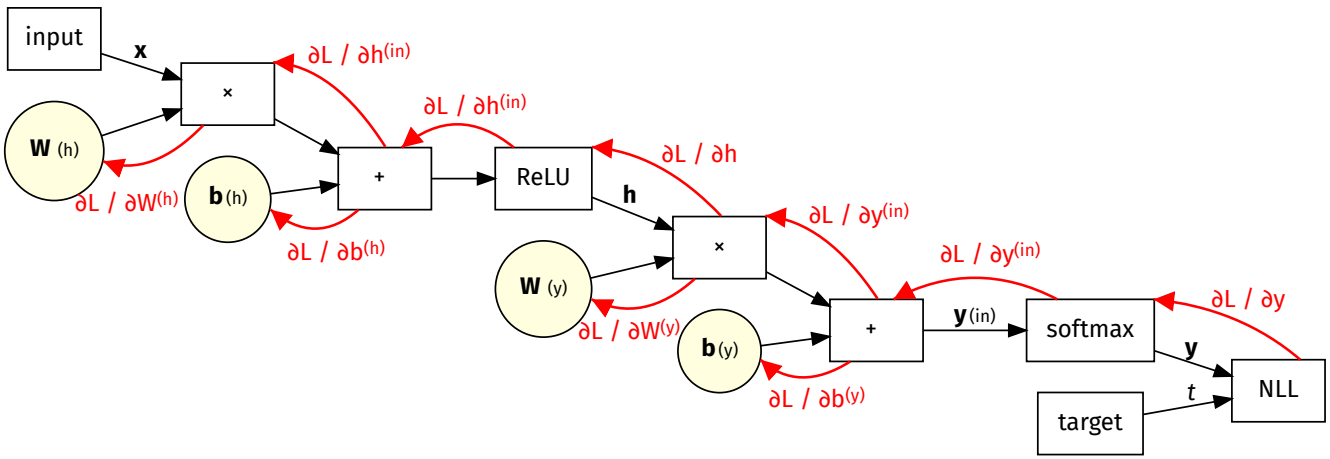
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x}$$

How to apply it? First, build a **computation graph** (example for a simple FFNN):



Backpropagation

...then apply the **chain rule** it in the backward direction:



Computing the gradient in practice

Modern deep learning frameworks (PyTorch, TensorFlow) have built-in **autograd** functionality that automatically computes gradients for us.

Stochastic gradient descent

Computing the gradient over the **entire dataset** at once is ~~expensive~~ impossible.

Idea

Instead of using all training examples, we iteratively compute the gradient on **small random subsets** of the training examples.

- **Stochastic gradient descent (SGD)**: our subset is of size 1 (=a single example).
- **Mini-batch SGD**: we a take a batch of examples (e.g. 32 examples).

Question

Is it useful to introduce some stochasticity into the training process?

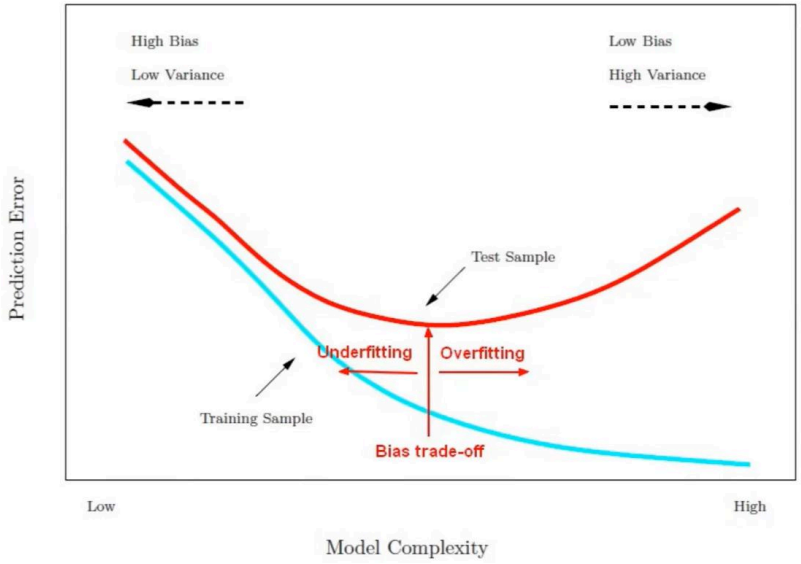
Learning rate

source: CS231n

The **learning rate** η controls how big the parameter update steps are.

Intuition about learning rate

- **Too high** → may overshoot the minimum, training diverges.
- **Too low** → converges very slowly, may get stuck in a bad local minimum.
- It is usually a small multiplier: 0.01, 0.001, …



Into the depths of deep learning

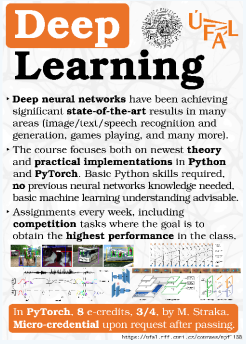
There are other concepts for NN training that we do not cover here, such as:

- **Weight initialization**: how do we initialize the model weights before training?
- **Regularization**: how do we prevent the model from overfitting to training data?
- **Optimizers**: is there are more efficient approach than SGD?

If you want to learn more...

See the MFF UK **Deep Learning course** (lectures 1-3, lecture recordings available):

https://ufal.mff.cuni.cz/courses/npfl138/2526-summer



Pretraining the Transformer

Supervised training?

source: Vaswani et al. (2017)

The original Transformer was trained **fully supervised on machine translation**:

- English → German (WMT 2014, 4.5M sentence pairs)
- English → French (WMT 2014, 36M sentence pairs)

> **Question**
>
> Where can we get that much data for other tasks?

We **do not have** that much data for other tasks 🥲

→ However, much of what the model was learning during the training was the **structure of the language** itself.

Self-supervised pretraining

Idea

If we train the model for predicting the words in a text, **any text is training data**!

W https://en.wikipedia.org/wiki/Large_language_model

A large **language** model (LLM) is a language model trained with self-supervised r

- The training regime where the labels come from the input itself is called **self-supervised learning**.
- We can use it for learning general language representations

  → ...maybe we will then need less data for the specific tasks?

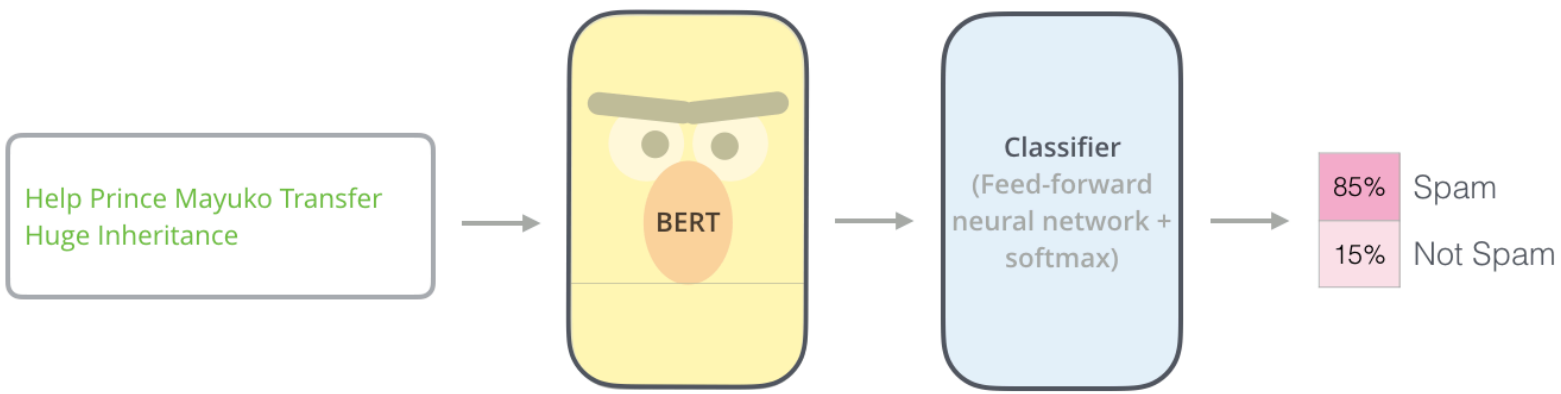
Pretraining a Transformer encoder

source: Illustrated BERT

Idea

We know the text representation is built in the Transformer **encoder**. Can we start pretraning the encoder alone?

Good representations might help us with classification tasks:

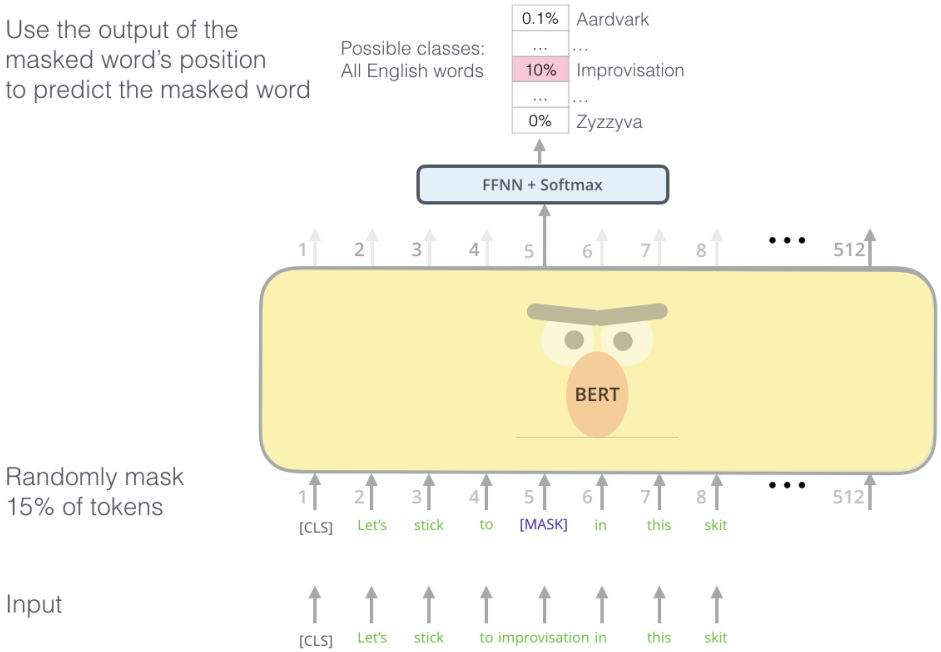


Pretraining a Transformer encoder

Our goal: Get a rich representation of the input text (one embedding per token).

- The resulting word embedding should incorporate the full sentence context (both **left and right**).
- Can we mask a word and let the model predict it?
  - Yes, but too slow → let's mask ~15% of words at a time.

→ the **masked language modeling** objective.



Pretraining a Transformer encoder

Our setup:

1 - Semi-supervised training on large amounts of text (books, wikipedia..etc).

The model is trained on a certain task that enables it to grasp patterns in language. By the end of the training process, BERT has language-processing abilities capable of empowering many models we later need to build and train in a supervised way.

Semi-supervised Learning Step

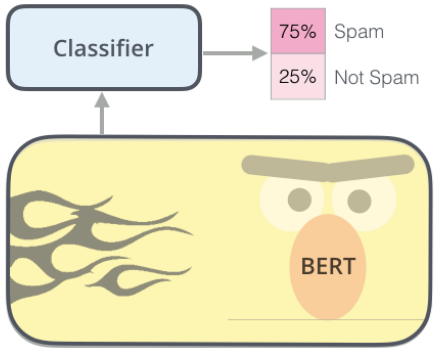
Model:

Dataset:

Objective: Predict the masked word (langauge modeling)

2 - Supervised training on a specific task with a labeled dataset.

Supervised Learning Step



Model: (pre-trained in step #1)

Dataset:

Email message	Class
Buy these pills	Spam
Win cash prizes	Spam
Dear Mr. Atreides, please find attached…	Not Spam

BERT: Pre-trained Transformer encoder

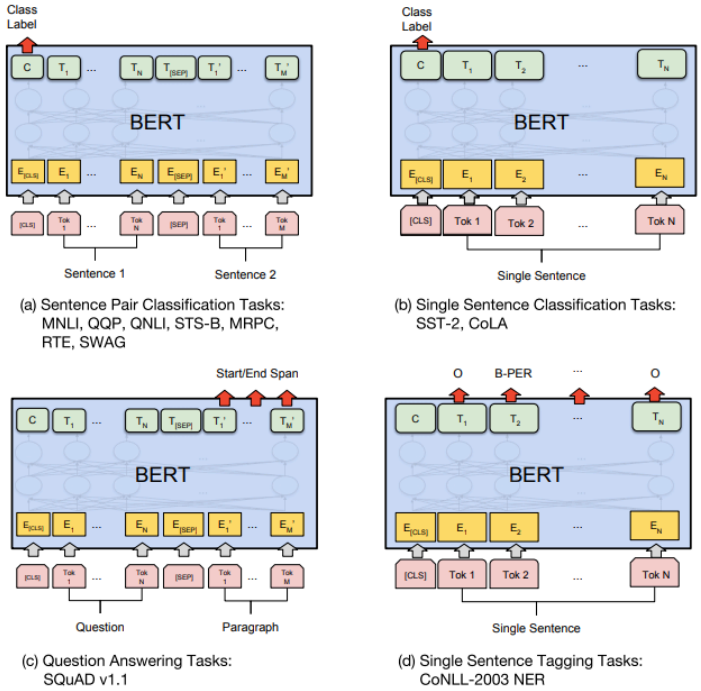
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

(Devlin et al., 2019) → breakthrough approach for many NLP tasks.

Info

The results BERT was getting were very cool – anything using BERT was suddenly **state-of-the-art**. No one was sure why.

This started a whole new research field called "BERTology".

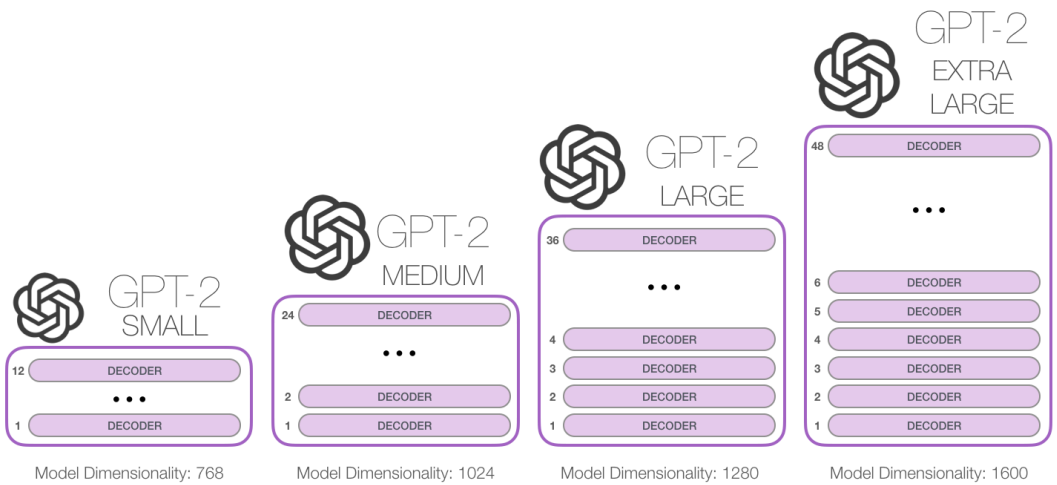


Pretraining a Transformer decoder

source: Illustrated GPT-2

Meanwhile in OpenAI...

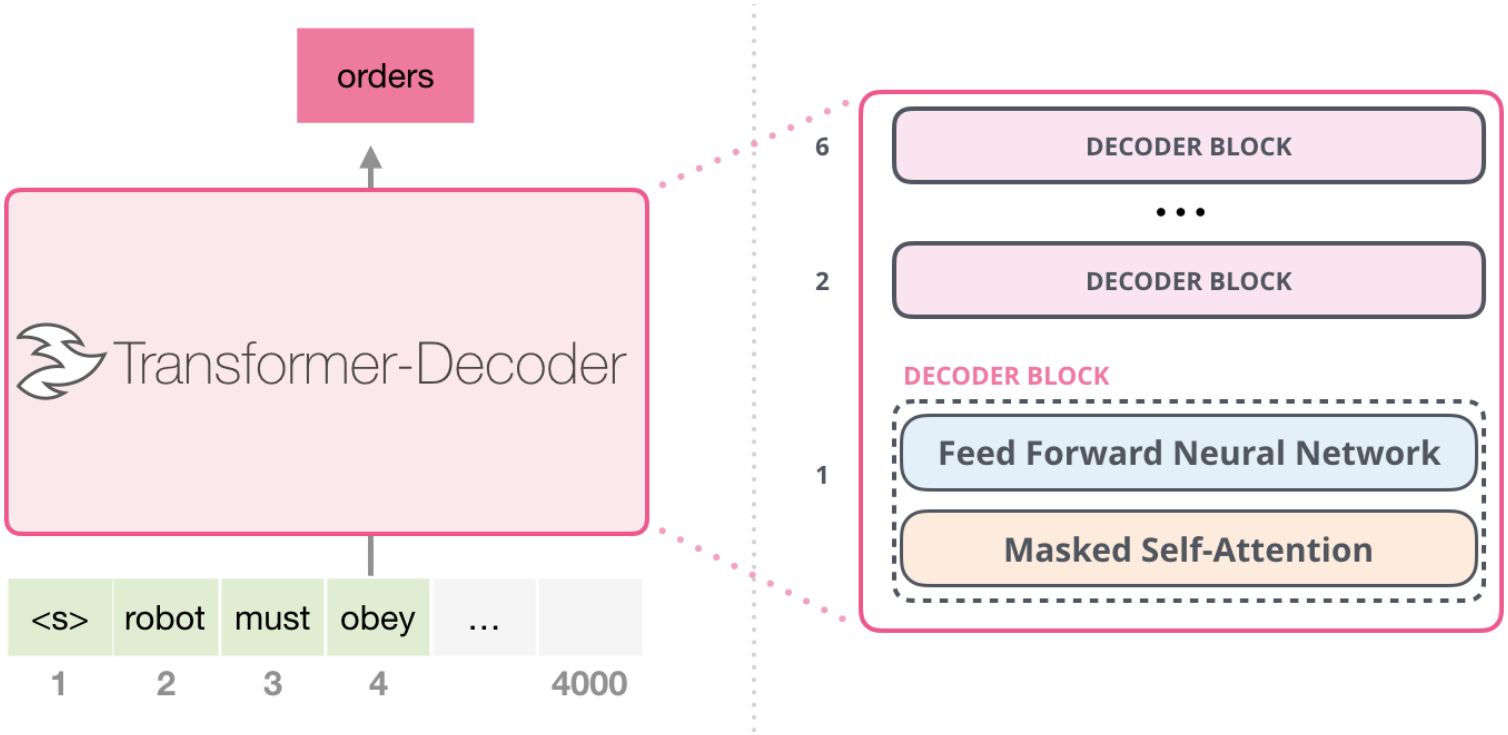
The encoder seems kind of useless. The **decoder** can represent the text on its own – and it can also **generate text**! Can we have more of that, please?



Pretraining a Transformer decoder

source: Illustrated GPT-2

Decoder-only block: only masked self-attention & FFNN, no encoder-decoder attention.



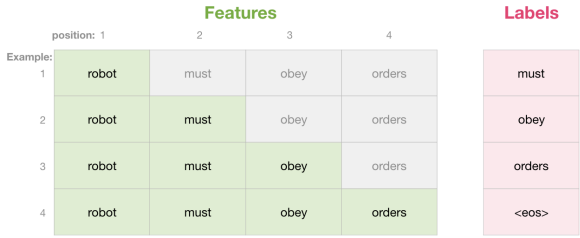
Causal language modeling

How to pretrain a decoder? Using **causal language modeling**: what we already know as "language modeling" or "next-word prediction task":

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1})$$

Attention mask

Masking the future tokens can be done using a triangular mask. That allows **parallel training**: we can compute the loss at all positions at once, while each position "pretends" it cannot see its future.

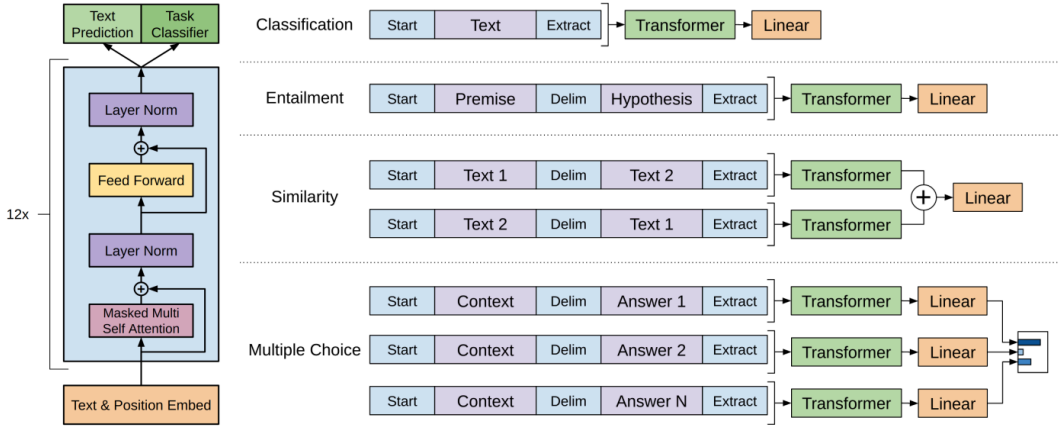


GPT: Pre-trained Transformer decoder

source: Radford et al. 2018

The "GPT-1" paper (Improving Language Understanding by Generative Pre-Training (Radford et al., 2018))

- 12 layers, 117M params, good results on downstream tasks.
- Published (pre-print + blogpost) in June 2018 → immediately overshadowed by BERT in October 2018.



GPT: Pre-trained Transformer decoder

source: OpenAI blog

GPT-2: Language Models are Unsupervised Multitask Learners (Radford et al., 2019)

Published as a pre-print and a blogpost on the OpenAI website in February 2019:

February 14, 2019 Milestone

Better language models and their implications

Read paper ↗ View code ↗

GPT: Pre-trained Transformer decoder

source: OpenAI blog

Examples of astounding (back then!) **consistency of text over several paragraphs**:

System Prompt (human-written)

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Model Completion (machine-written, 10 tries)

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science.

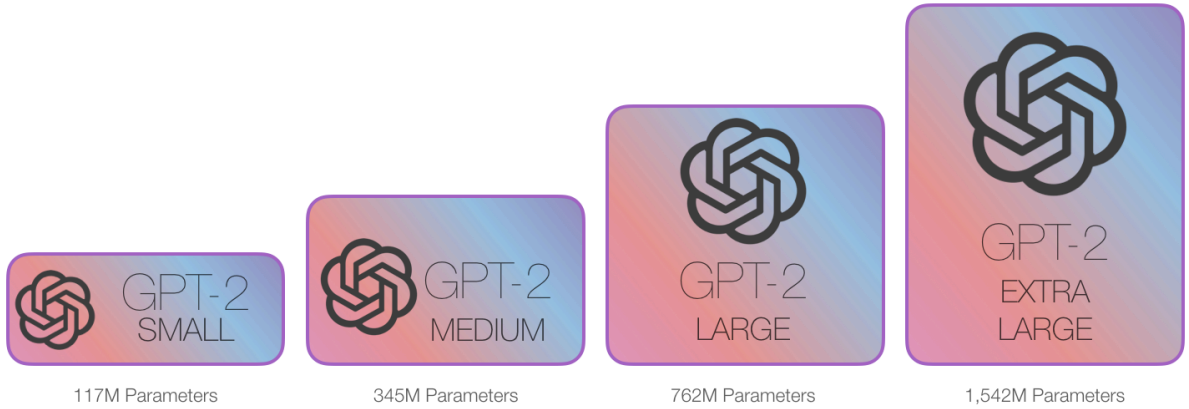
(...7 more paragraphs...)

However, Pérez also pointed out that it is likely that the only way of knowing for sure if unicorns are indeed the descendants of a lost alien race is through DNA. "But they seem to be able to communicate in English quite well, which I believe is a sign of evolution, or at least a change in social organization," said the scientist.

GPT: Pre-trained Transformer decoder

GPT-2 release strategy

At first, OpenAI decided to release only the **smallest model** version *"due to concerns about large language models being used to generate deceptive, biased, or abusive language at scale"*. They released all the models within ~6 months.



Encoder-decoder pretraining

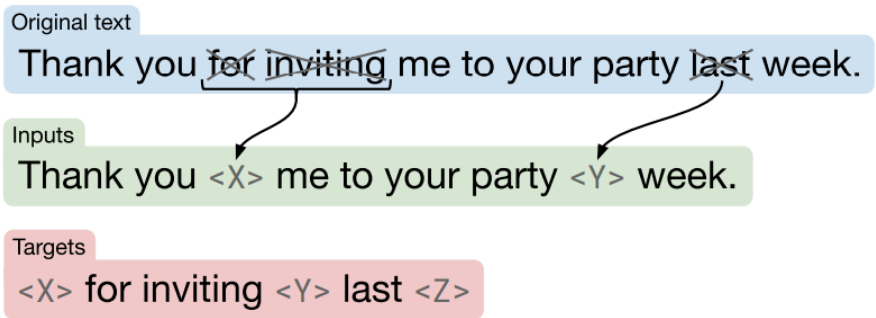
source: Raffel et al. (2020)

Question

Can we also **pre-train** the full encoder decoder model?

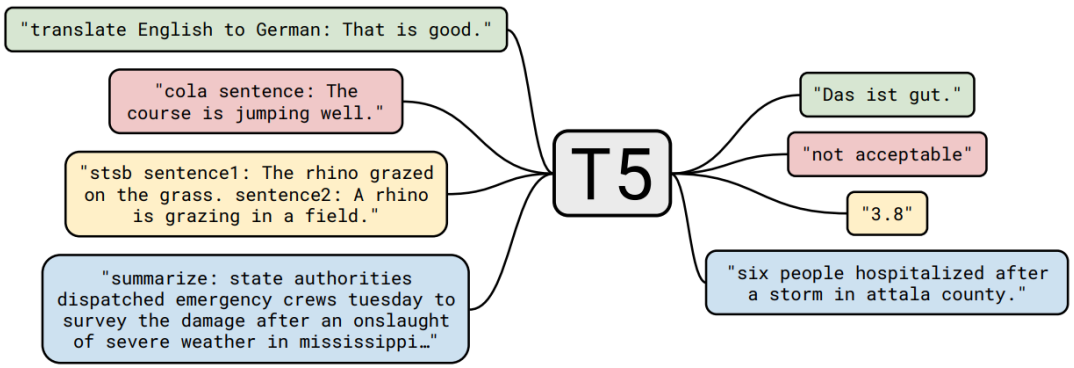
Span corruption (→ the "denoising" objective):

- The model needs to predict continuous spans in the sequence.
- Combines the benefits of bidirectional encoding with autoregressive generation.

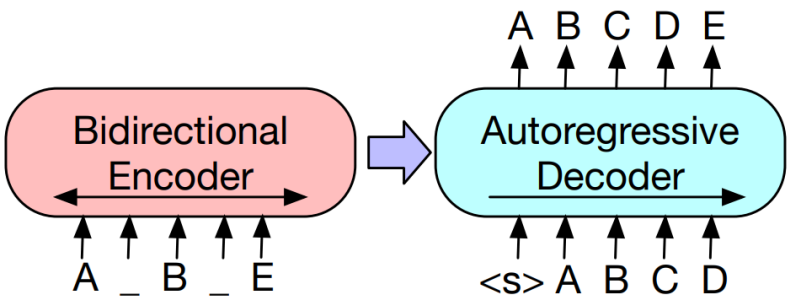


T5 & BART: Pre-trained Transformer encoder-decoders

T5 (Raffel et al., 2019) - Google:



BART (Lewis et al., 2019) - Microsoft:



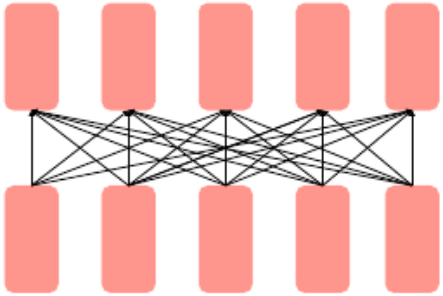
- Very similar models (slightly different training objectives & training data).
- State-of-the-art on **seq2seq tasks** until the rise of large language models.

What we covered

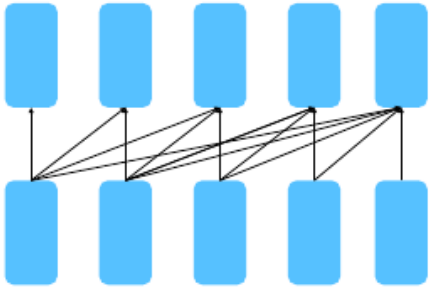
source: CSE 517 / CSE 447

Three variants of pre-trained Transformer models:

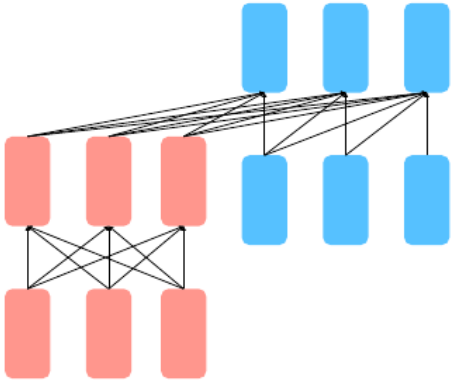
Encoder-only (BERT)



Decoder-only (GPT)



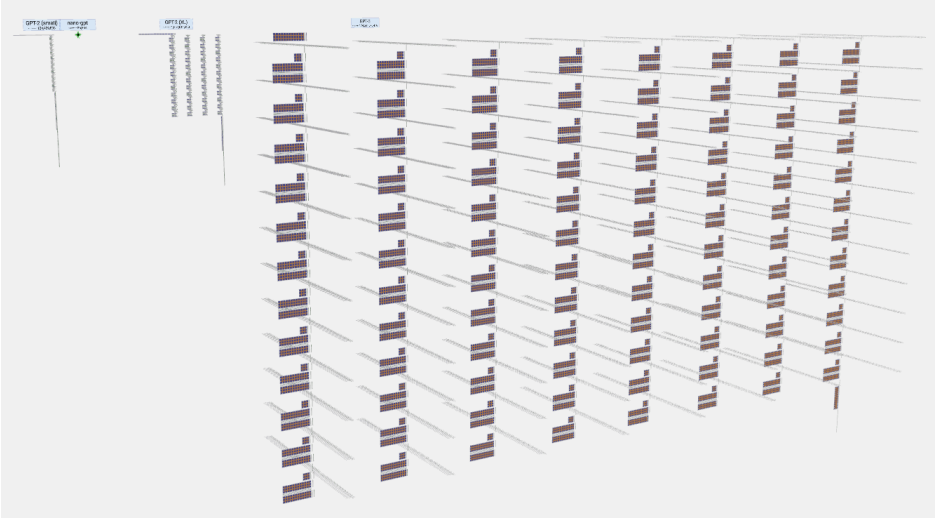
Encoder-decoder (T5, BART)



GPT-3

Meanwhile in OpenAI...

GPT-2 was kinda good. Let's see what happens if we make it **100x as big**!



source: llm-vis

GPT-3

source: Brown et al. 2020

GPT-3: Language Models are Few-Shot Learners (Brown et al. 2020)

- **175B parameters** (the largest GPT-2 model was 1.5B parameters)
- Trained on a mix of Common Crawl, books, Wikipedia.

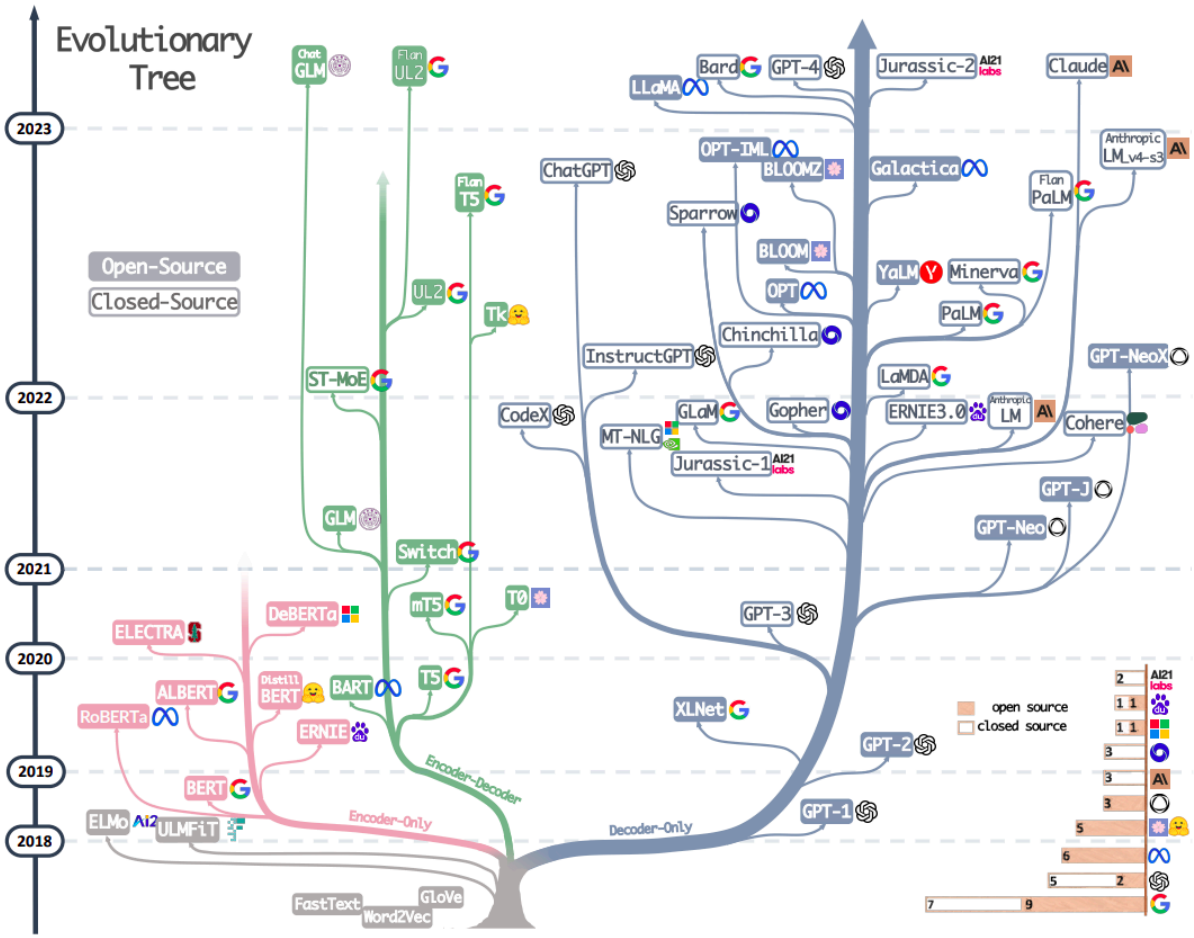
Biggest surprise: In-context learning

The model could suddenly learn a task from a few examples provided in the input prompt (without any gradient updates).

- *Zero-shot*: Task description only, no examples.
- *Few-shot*: Task description + a few examples.

The LLM evolutionary tree: how it continued

source: Yang et al. 2023



Legend:

- Encoder models
- Encoder-decoder models
- Decoder models

Finetuning and alignment

Where are we?

Model stages:

random neural network

1

"autocomplete on steroids"

base / foundational model

2

assistant

instruction-tuned model

3

helpful assistant

Training stages:

1 **Pre-training** 🌍

Prague is the capital of Czechia (...)

2 **Instruction tuning** 💬

user: What is the capital of Czechia?

assistant: **Prague**

3 **Human preference optimization** 🧑‍⚖️

user: What is the capital of Czechia?

answer #1: Prague.

answer #2: The capital of Czechia is Prague.

Problems with base models

Base models are just an **"autocomplete on steroids"** → the following can happen:

Input	*Explain the moon landing to a 6 year old.*
Pre-trained model	*Explain why the sky is blue to a 6 year old. Explain why planes can fly to a 6 year old. Explain how to build a car to a 6 year old (...)*
What we want instead	*Many years ago, people wanted to explore the Moon. So they built a huge rocket called Saturn V and sent astronauts in it. (...)*

Question

Can this problem be fixed with few-shot prompting?

Finetuning

👩‍🍳 Recipe for finetuning a language model:

1. Start from a **pretrained "base" language model**.
2. Prepare a dataset of **(input, output) pairs** for the target task.
3. Continue training with the **same objective** (cross-entropy on the output tokens).

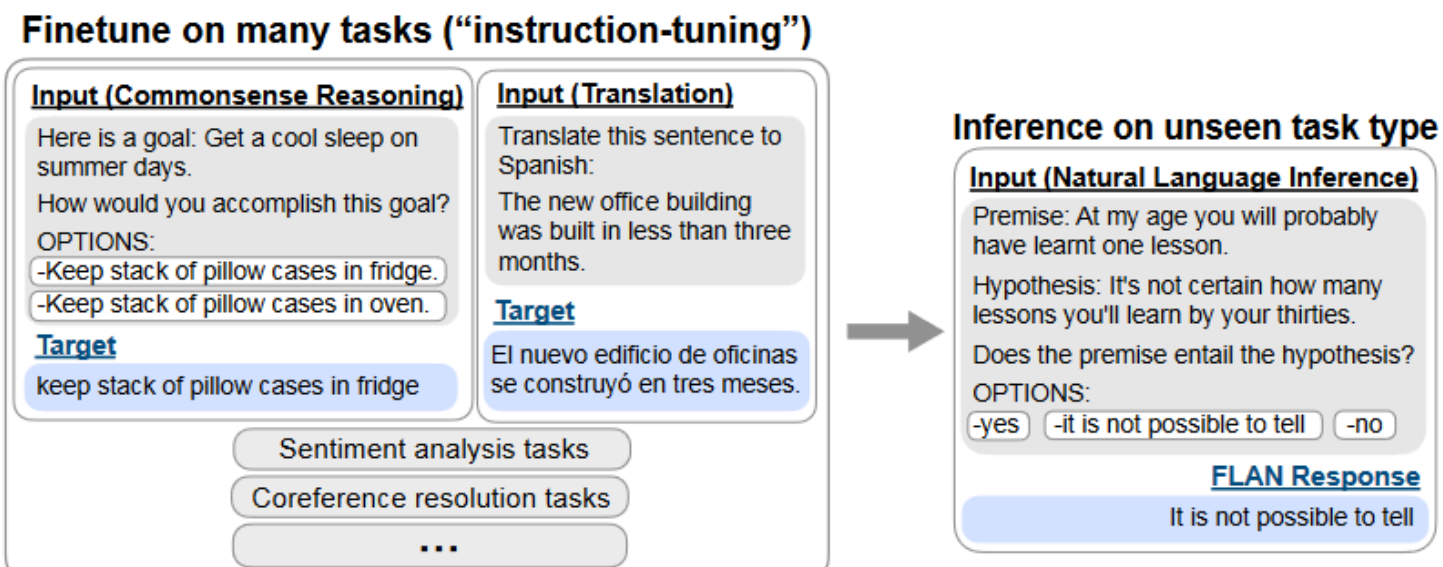
> **Why does this work?**
>
> The pretrained model has already learned rich language representations. Finetuning only needs to **slightly adjust** the parameters to fit the new task. This is much more data-efficient than training from scratch.

Instruction tuning

source: Wei et al. 2022 (FLAN)

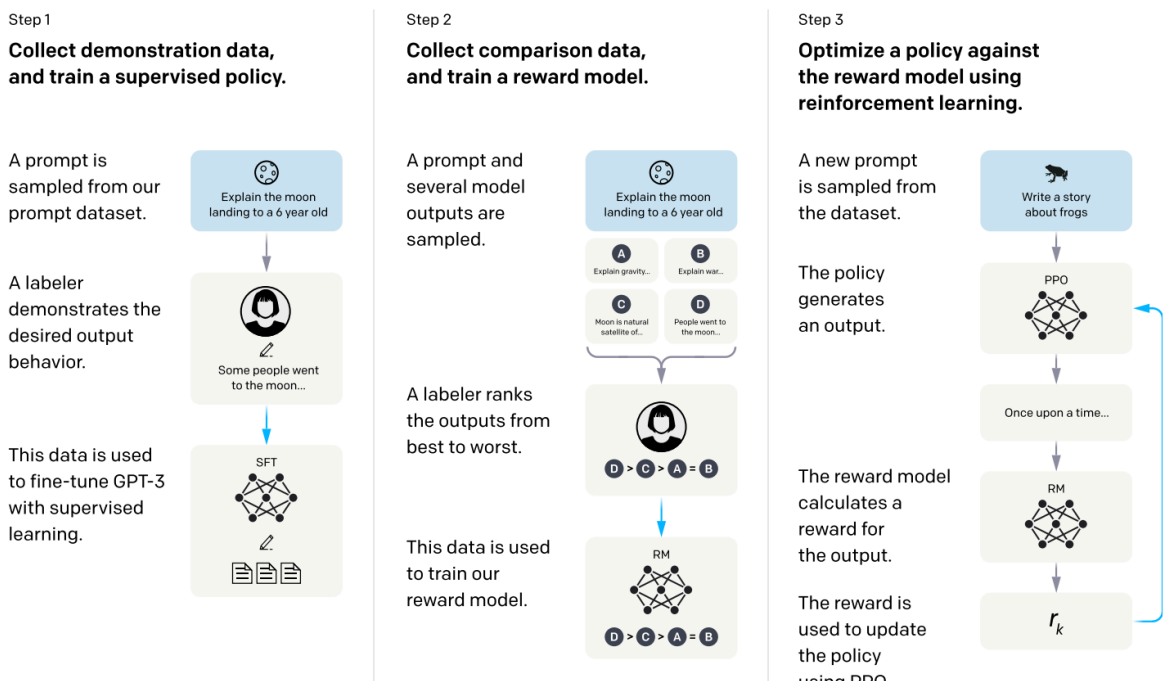
Instruction-tuning: finetuning the model to **follow instructions**:

- **Input**: a natural language description of the task + the actual input.
- **Output**: the expected response.



InstructGPT

InstructGPT: [Training language models to follow instructions with human feedback (Ouyang et al., 2022)](#) → released ~6 months before ChatGPT.



Problems with instruction-tuned models

Even the instruction-tuned model may still produce **harmful or toxic** content, be **unhelpful**, or refuse reasonable requests.

Idea

Let people score which outputs are better, train the models to prefer these.

Question

Can we just apply supervised finetuning again?

Reinforcement learning from human feedback (RLHF)

🧑‍⚖️ **Step 0 – Gather human annotators.**

⚖️ **Step 1 – Collect pairwise rankings**:

- Generate two outputs (y_1, y_2) for the same prompt x.
- Have annotators rank them: $r(x, y)$.

⚙️ **Step 2 – Align the model using reinforcement learning**

- Optimize the LLM using the ranking $r(x, y)$ as a reward signal.
  - Include a penalty to prevent drifting too far from the original model:

$$\mathcal{L}_{\text{RLHF}} = -\mathbb{E}[r(x, y)] + \beta \cdot \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$$

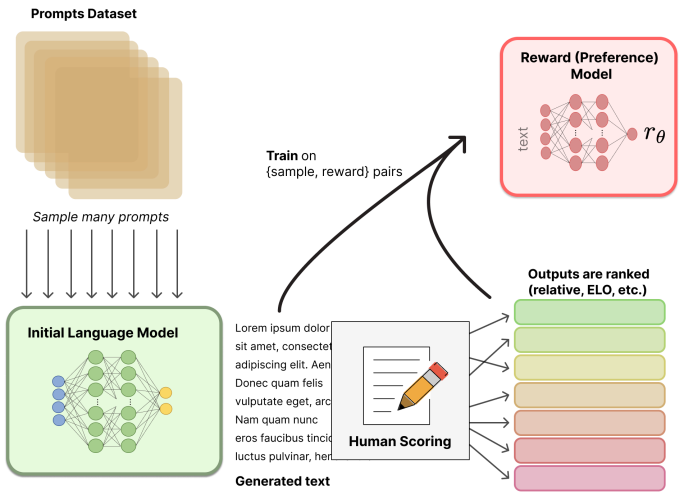
RLHF - Reward model (→RLAIF)

source: HF blog

Idea

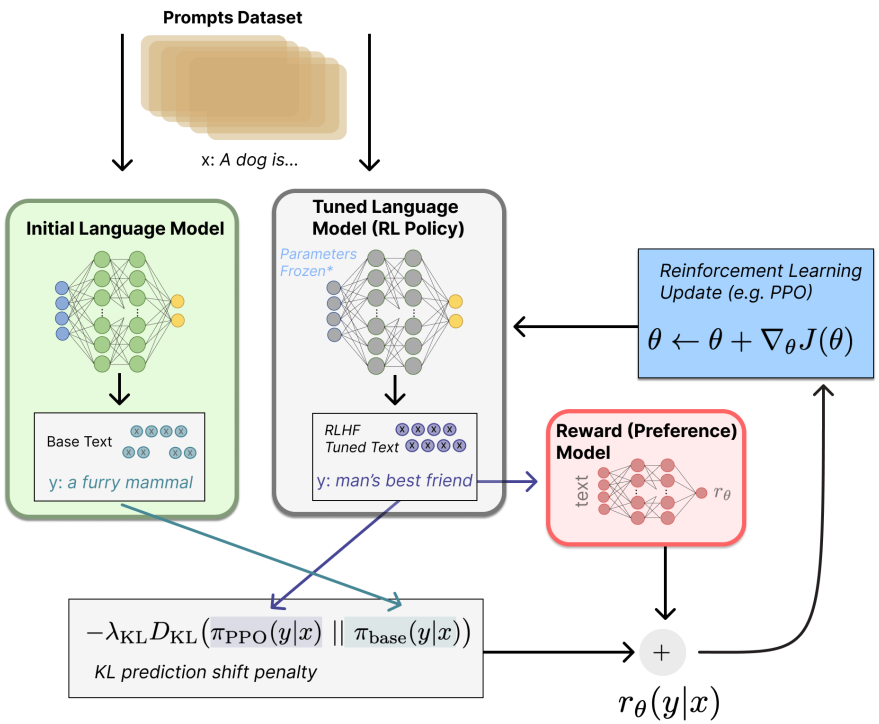
Human annotators are costly and slow. Can we replace them with a **model**?

Yes: we can train a **reward model** on the collected human feedback:



RLHF - Reward model (→RLAIF)

...and then **replace human annotators** with the reward model:



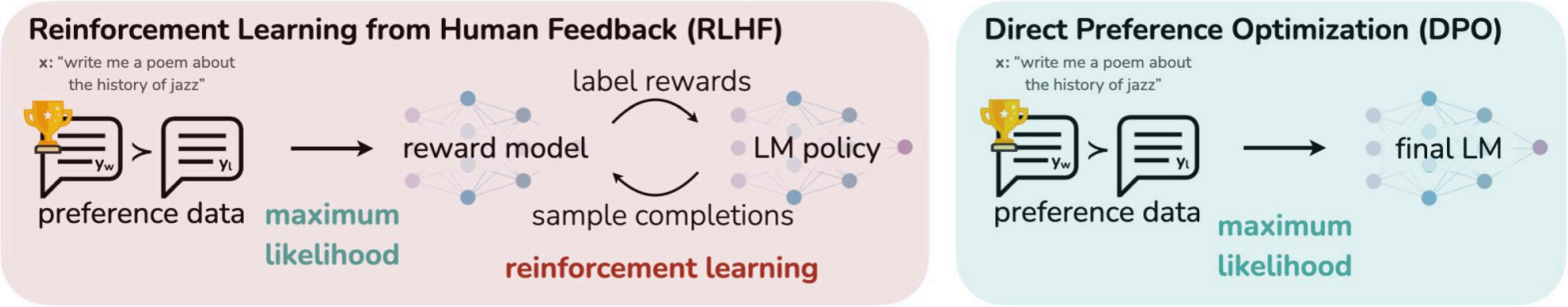
Direct preference optimization (DPO)

source: Rafailov et al. 2023

DPO skips the reward model and RL entirely.

Given pairs of (preferred, dispreferred) responses, DPO directly optimizes a **special loss function** (where y_w = preferred response, y_l = dispreferred response):

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right)$$



Summary

Summary

What we covered today

- **Training NNs**: loss functions, gradient descent, SGD, backpropagation, learning rate.
- **Pretraining**: self-supervised objectives (CLM, MLM, span corruption).
- **Pretrained models**: BERT, GPT, T5 and their variants; the LLM timeline.
- **Supervised finetuning**: adapting pretrained models to specific tasks; instruction tuning.
- **Alignment**: RLHF, DPO – making models helpful and safe.

Links

- BERT: Pre-training of deep bidirectional Transformers – Devlin et al. (2019)
- Language models are few-shot learners (GPT-3) – Brown et al. (2020)
- Exploring the limits of transfer learning (T5) – Raffel et al. (2020)
- Training language models to follow instructions (InstructGPT) – Ouyang et al. (2022)
- Direct preference optimization (DPO) – Rafailov et al. (2023)
- Illustrating RLHF – Hugging Face blog
- An overview of gradient descent optimization algorithms – Ruder (2016)
- The Illustrated BERT – Jay Alammar